

# Фаза 3.2: Детерминированный RAG и Изолированные Подграфы Знаний (Deterministic Sub-graph RAG)

**Авторы:** Казбек & AI Engineering Lead (CAE Data LLC R&D)

**Статус:** Архитектурная спецификация и реализация (Implementation Spec)

## 1. Проблема: «Векторная слепота» и Кросс-доменные галлюцинации

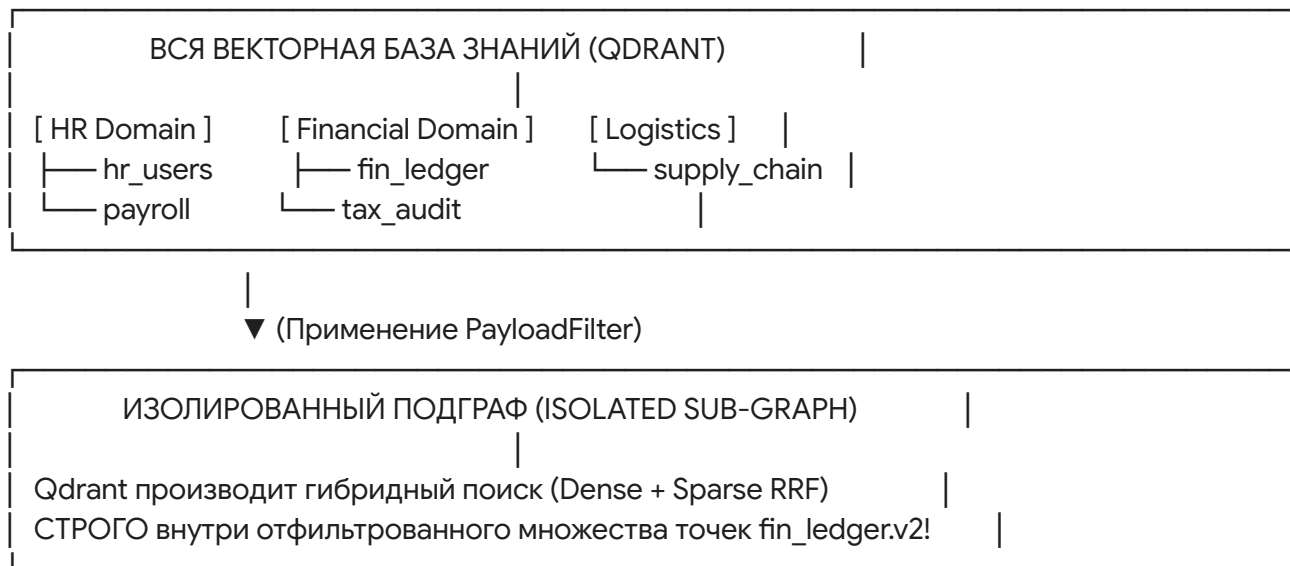
В традиционных RAG-системах поиск производится по всей коллекции векторов без разбора. Это порождает опасный класс ошибок в DataOps:

1. **Кросс-доменная наводка (Cross-Domain Pollution):** Ошибка в колонке status финансовой таблицы `fin_ledger` вытаскивает правило качества для колонки status из таблицы пользователей `hr_users`. Модель пытается применить логику пользовательских аккаунтов к бухгалтерским проводкам.
2. **Версионный конфликт (Schema Drift):** Если в базе есть правила для схемы v1.0 и v2.0, чистый векторный поиск с высокой вероятностью подтянет старый устаревший контракт.
3. **Нарушение изоляции клиентов (Tenant Isolation):** В практике CAE Data LLC смешивание метаданных разных клиентов или юрисдикций категорически недопустимо.

## 2. Решение: Qdrant Deterministic Payload Filtering

Мы внедряем двухэтапную схему Retrieval:

$$\text{Search Space} = \text{Collection} \cap \text{PayloadFilter}(\text{tenant}, \text{domain}, \text{table}, \text{version})$$



### Ключевые поля полезной нагрузки (Metadata Payload Schema):

- tenant\_id (str): Идентификатор клиента / проекта (например, cae\_data\_llc).
- domain (str): Бизнес-домен (finance, hr, regulatory).
- table\_name (str): Имя целевой таблицы данных (salaries\_stage).
- schema\_version (str): Версия схемы контракта (v1.0).
- criticality (str): Критичность поля (CRITICAL, NON\_CRITICAL).

## 3. Математика поиска с фильтром в Qdrant

При выполнении `query_points()` в Qdrant, векторный индекс HNSW и разреженный индекс BM25 пересекаются с битовым массивом фильтра:

$$RRF\_Score(d) = \sum_{m \in M} \frac{1}{k + r_m(d)} \quad \forall d \in \{x \in \mathcal{D} \mid \text{MatchFilter}(x) = \text{True}\}$$

Точки, не прошедшие `query_filter`, полностью исключаются из математического расчета рангов еще **до** стадии слияния RRF, сохраняя скорость отклика на уровне микросекунд.

## 4. Сценарий работы в Графе Оркестратора

1. При возникновении сбоя оркестратор определяет контекст таблицы (`table_name`, `domain`).
2. В узел `node_ai_analyst` передается сформированный `models.Filter`.
3. Qdrant ищет совпадения **только** среди правил этой таблицы и домена.
4. Если точно совпавших правил в данном изолированном подграфе нет — система явно фиксирует отсутствие доменного правила, предотвращая подтягивание чужого мусора.